

Project: Flood Monitoring and Management System (FMMS)

Version	2.0.0
---------	-------

Date	08/01/2025
Team Name/ID	Design Syndicate

Team Members	Matriculation number	E-mail address
Md Abdus Samad	301148	mdabdus.samad@student.univaq.it
Salim Kamaludeen Lawal	301134	salimkamaludeen.lawal@student.univaq.it

Table of Contents

Challenges/Risk Analysis.....	4
State of the Art (SOTA) Analysis	5
StormSense.....	5
Global Flood Monitoring (GFM) - Copernicus.....	6
FLOODWALL: A Real-Time Flash Flood Monitoring and Forecasting System Using IoT	7
Requirements Refinement	8
Functional Requirements (FR).....	8
Non-Functional Requirements (NFR).....	10
Informal Description of your system and its Software/System Architecture	11
Architectural styles.....	13
Microservice.....	13
Event-Driven	13
Architectural Tactics.....	14
Performance and Scalability	14
Redundancy	14
Usability.....	14
Reliability	15
Load Balancing	15
Adaptive System Prioritization	15
Security.....	15
Sub-systems view	16
IoT Data Collection Subsystem (1).....	16
Gateway Subsystem (2)	17
Data Processing Subsystem (3).....	17
Weather Data Integration Subsystem (4).....	17
Alert Management Subsystem (5).....	17
Data Visualization Subsystem (6).....	17
Components, Connectors, and Interfaces View	18
Components.....	18
Connectors	19
Interfaces	19
	2

System boundaries.....	21
CAPS view	21
CUPCARBON view	22
Architecture Decision Record	23
Architecture Design Decisions [for deliverables D2]	24
Architectural Design Decision 1 – Selecting Sensors.....	24
Architectural Design Decision 2 – Selecting the Message Bocker	25
Architectural Design Decision 3 – Selecting Gateway	26
Architectural Design Decision 4 – Deployment and Scalability	27
Pitch of your proposal [for deliverable D2].....	28
Prototype Description [for deliverable D2].....	28
What does this prototype demonstrate?.....	28
How would this prototype satisfy functional and non-functional requirements and bring the required quality characteristics?	29
Implementation Architecture [only for Deliverable D2].....	30
“Demonstration” of your architecture compliance to Functional and Non-Functional Requirements [only for Deliverable D2]	31
Summary [only for Deliverable D2].....	34

Challenges/Risk Analysis

[for both deliverables D1 and D2]

Risk ID and description	Date the risk is identified	Date the risk is resolved	Explanation of how the risk has been managed
R01: Ensure accurate and timely flood warnings	30/10/2024	02/11/2024	Instead of relying on complex machine learning models, we prioritize accuracy and timeliness by integrating weather data APIs and adjusting sensor frequency dynamically to provide early flood warnings. This ensures practical forecasting aligned with the system's goals of reliability and simplicity.
R02: Broad coverage and cost-efficiency for data collection	30/10/2024	30/10/2024	To address this, we focus on selecting a communication strategy that balances coverage, cost, and energy efficiency. LoRaWAN was chosen due to its proven effectiveness in remote, large-scale deployments. This aligns with the system's goal of broad accessibility.
R03: Seamless communication between sensors and the system	30/10/2024	03/11/2024	The challenge is to maintain consistent data transmission without introducing unnecessary complexity. We have implemented the sensors to communicate directly with gateways, eliminating additional intermediaries and ensuring efficient edge processing.
R04: Choice of user interface for flood monitoring	10/11/2024	10/11/2024	To meet diverse user needs, the system supports both mobile and web platforms using a hybrid approach. This ensures accessibility, usability, and alignment to provide real-time, actionable insights.
R05: How can the system ensure seamless user management and sensor data	15/11/2024	17/11/2024	To balance simplicity and efficiency, we consolidated user and sensor data into a single database. This prevents conflicts and ensures consistent access for the alerting service and API, with read and write operations optimized to avoid bottlenecks. The decision aligns with the goal of streamlined data management while maintaining operational reliability.

State of the Art (SOTA) Analysis

[for deliverable D1]

StormSense

StormSense is an IoT-driven flood monitoring and prediction system designed to enhance coastal resilience by providing real-time data on water levels and storm surges. It integrates a network of sensors to monitor environmental parameters during storms, offering precise flood forecasts and early warnings. By collaborating with local governments and communities, StormSense supports rapid, informed responses to impending flood threats. Its advanced predictive models and real-time monitoring enhance safety and preparedness in vulnerable coastal areas.

What I Like
It integrates with other local and regional flood monitoring systems, contributing to a more comprehensive view of flood risks.
The use of AWS IoT handles large volumes of data and supports extensive geographic coverage.
Uses predictive models to anticipate flood risks, improving response times for impending flood threats.
Built with APIs and standardized protocols, enabling integration with existing emergency response systems, weather data sources, and GIS platforms for comprehensive situational awareness.
What I do not like
The use of advanced sensors and integration with predictive models requires significant initial investment.
Although effective in coastal areas, scaling across larger regions or non-coastal flood-prone areas may require additional calibration and system resources.
What I plan to reuse, from an architectural viewpoint
Like StormSense, FMMS could benefit from real-time visual dashboards, especially those that allow managers to view environmental parameters by geographic area.
StormSense's architecture supports various user roles, from local to regional managers, a model FMMS could be reused to allow different management levels access to relevant data based on geographic responsibility.

Source:https://www.vims.edu/people/loftis_id/StormSense/

Global Flood Monitoring (GFM) - Copernicus

Global Flood Monitoring (GFM) is a real-time tool developed by the European Commission's Joint Research Centre to monitor flood risk worldwide. It combines hydrological and meteorological data to predict potential floods up to 30 days in advance. GFM offers near-real-time flood mapping, early warnings, and risk assessment through satellite and model data integration. The system is particularly valuable in identifying high-risk flood areas across diverse regions, enabling timely responses to mitigate damage.

What I Like
The system's alerting function is geo-specific, sending alerts based on precise geographic locations affected by flood risks. Automated alerts to stakeholders, paired with map visualizations, ensure that only relevant users are notified, reducing information overload.
GFM operates on a cloud-based architecture, allowing it to scale dynamically based on demand, which is ideal for handling sudden increases in data processing during flood events. This can support large-scale data processing and accommodate high traffic loads.
GFM integrates satellite, meteorological, and hydrological data, enhancing flood detection and response accuracy.
What I do not like
GFM relies primarily on remote sensing and does not integrate with physical sensors, which can provide more precise, area-specific data.
What I plan to reuse, from an architectural viewpoint
The user-friendly visualization and access controls in Glo could inspire FMMS's interface, ensuring that managers at different levels can access relevant data and alerts specific to their geographic areas.
GFM employs advanced data fusion algorithms to integrate data from different sources, such as meteorological models and satellite imagery. FMMS could adopt similar techniques to combine real-time sensor data with historical data and external meteorological information for enhanced accuracy.

Source: <https://global-flood.emergency.copernicus.eu/technical-information/glofas-gfm/>

FLOODWALL: A Real-Time Flash Flood Monitoring and Forecasting System Using IoT

FLOODWALL is a real-time flash flood monitoring and forecasting tool using IoT technologies. It collects hydrological data, like water flow and discharge, along with meteorological data, such as temperature and rainfall, to assess flood risks. The system employs an LSTM model to analyze and classify flood events into alert levels, ranging from "no alert" to "red alert," based on real-time data. This setup provides timely alerts to users and authorities, improving emergency preparedness in flood-prone areas

What I Like
The use of physical sensors in FLOODWALL is prioritized for their proximity-based accuracy, providing precise real-time data essential for reliable flood forecasting. This approach enhances data quality compared to remote data sources, supporting more responsive and accurate flood monitoring.
The integration of cloud technology in FLOODWALL is highly beneficial, enabling centralized data storage, real-time access, and scalability. This approach enhances the system's efficiency and allows seamless monitoring and analysis across different locations.
What I do not like
The use of an LSTM model for forecasting in FLOODWALL may not be the most cost-effective choice, as similar predictive insights could be achieved using external weather APIs. Leveraging a weather API could simplify the system, reduce computational demands, and lower costs while maintaining effective flood forecasting capabilities.
The inclusion of intermediate devices like Arduino in FLOODWALL introduces additional components that may not be essential, adding complexity and potential costs without significant benefits. Streamlining the system to reduce reliance on such devices could enhance efficiency and simplify implementation.
What I plan to reuse, from an architectural viewpoint
Cloud technology because it offers scalable, reliable data processing and storage capabilities critical for managing high-frequency sensor data from multiple locations. Leveraging an established cloud infrastructure ensures real-time data accessibility, efficient resource allocation, and enhanced performance across the system

Source: C. Prakash, A. Barthwal and D. Acharya, "FLOODWALL: A Real-Time Flash Flood Monitoring and Forecasting System Using IoT," in IEEE Sensors Journal, vol. 23, no. 1, pp. 787-799, 1 Jan.1, 2023, doi: 10.1109/JSEN.2022.3223671.

Requirements Refinement

[for deliverable D1. If needed, refined in D2]

Functional Requirements (FR)

ID	Description	Priority	Is Architecturally Significant?
FR1	The system must process, and display collected data in real-time, providing continuous updates on environmental parameters	HIGH	YES High-performance architecture to ensure low latency in data handling and visual updates
FR2	The system must provide automatic notifications when sensor values exceed defined safety thresholds, with clear visual indicators	HIGH	YES Robust messaging and event-handling capabilities
FR3	The system must issue immediate warnings for situations, escalating alerts to higher management levels	HIGH	YES Robust messaging and event-handling capabilities
FR4	Sensors must increase data transmission frequency when critical values are detected to provide more granular data during emergencies	MEDIUM	NO
FR5	The system must notify users of sensor malfunctions, prioritizing alerts based on the criticality of the sensor and availability	HIGH	YES Requires prioritization logic and integration with notification mechanisms.
FR6	Each sensor must periodically measure and transmit the following environmental parameters: • Water levels • River flow velocity • Cumulative rainfall • Soil saturation • Wind speed and direction	MEDIUM	NO
FR7	Each sensor must have a unique identifier and be associated with a specific geographic area prone to hydrogeological risks	LOW	NO
FR8	The system must enable area managers to add new sensors to the system	LOW	YES Requires architectural adjustments to fit new sensors in the system
FR9	The system must allow regional managers to add new area managers	LOW	NO
FR10	The system must allow regional managers to add a new area	LOW	NO

FR11	Each sensor must transmit its operational status along with the environmental data	LOW	NO
FR12	Managers must be able to select and monitor specific areas and associated sensors through a geographic information system (GIS) interface	MEDIUM	NO
FR13	The system must integrate a weather information API to dynamically adjust sensor data collection frequency and proactively alert users of potential environmental risks	MEDIUM	YES Integration with an external weather information API requires a flexible architecture that can support API interactions
FR14	The system must support multiple levels of user access based on geographic area • Area Manager: Monitors and manages a specific zone • Regional Manager: Accesses an aggregated view	LOW	NO

Non-Functional Requirements (NFR)

ID	Description	Priority	Is Architecturally Significant?
NFR1	High Throughput Performance: The system architecture must handle at least 150,000 messages per minute, ensuring timely data processing and alerting	HIGH	YES With 150,000 messages per minute, this requirement is central to system performance and throughput design
NFR2	Reliability: The system must handle sensor malfunctions, providing redundancy and escalating alerts based on sensor criticality	HIGH	YES Fault tolerance for sensor malfunctions demands a resilient system architecture with redundancy and error-handling mechanisms
NFR3	Availability: The system must support continuous operation and access by up to 50 simultaneous users	HIGH	YES Continuous operation with simultaneous user access impacts infrastructure choices, particularly in terms of system load balancing and redundancy
NFR4	Security: The system must implement security measures to protect data integrity and prevent unauthorized access	MEDIUM	YES Security requirements affect architectural decisions related to data storage, access control, and encryption
NFR5	Fault Tolerance: The system must provide backup sensors in case of battery consumption and malfunctions	HIGH	YES Fault tolerance for (physical) sensor malfunctions demand a resilient system architecture with backup strategies

Informal Description of your system and its Software/System Architecture

[for deliverable D1, refined in D2]

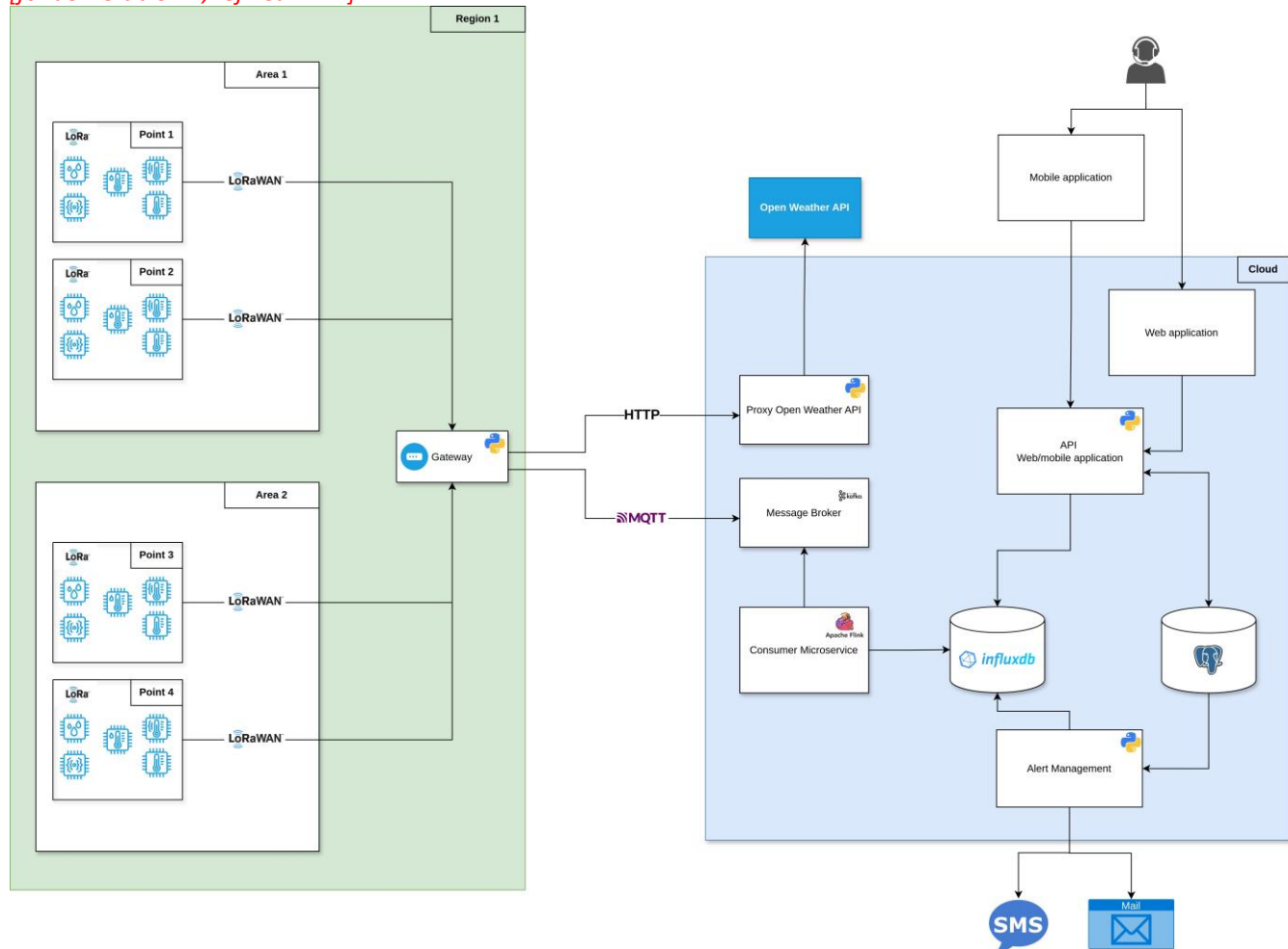


Figure 1 Informal Architecture View

The system initiates field operations by deploying uniquely identified LoRaWAN-enabled sensors at multiple strategic locations within flood-prone areas. These sensors continuously monitor critical environmental parameters, including water levels, river flow velocity, rainfall, soil saturation, and wind conditions. Each deployment site includes primary and backup sensors to ensure redundancy (**FR6, FR7, NFR2, NFR5**).

Sensor data, along with operational status information (e.g., battery levels, connectivity), is collected at regular intervals or triggered by specific events such as environmental threshold exceeded, sensor alerts, malfunction alerts etc. The collected data is transmitted to nearby gateways equipped with edge computing capabilities. The edge devices preprocess and filter the data locally to reduce latency and bandwidth usage before forwarding it to the cloud infrastructure for further analysis (**F11, NFR1**).

In the cloud, the data is routed through a message broker and consumed by a processing service that sits in between message broker and influxDB database (**FR1**). This ingestion process operates

continuously, with real-time or near-real-time updates depending on the frequency of data transmission from the gateways.

The alert management microservice periodically queries the database to evaluate the incoming data against predefined thresholds and operational rules (**FR2**). When the system detects conditions indicative of a [critical situation \(e.g., exceeding water level thresholds, sensor malfunctions, imminent flood threat\)](#), it immediately generates alerts. [The alert generation logic is rule-based, differentiating responses based on the nature of the issue:](#)

For low sensor battery levels, an email notification is sent to the area manager.

If a backup sensor is activated due to primary sensor failure or if both sensors fail, an SMS alert is sent to the area manager (**FR5, NFR2**).

For flood-related thresholds, SMS alerts are sent to both area and regional managers if the threshold is breached, whereas warning emails are sent to the area manager when thresholds approach critical levels.

Alerts are also visualized in real time on a user interface accessible via both web and mobile applications. The interface enforces role-based access controls: area managers can view data for their designated areas, while regional managers can view data across multiple areas or the entire region (**FR1, FR2, FR12, FR14**). The system also allows regional managers to configure new areas and assign corresponding area managers, while area managers can manage sensors within their respective areas (**FR8, FR9, FR10**).

The system dynamically adjusts the data collection frequency of the sensors based on predictive analytics that integrate external weather forecasts fetched via the Open Weather API. By combining real-time sensor data with forecast data, the system enhances its ability to anticipate and mitigate flood risks in advance (**FR4, FR13, NFR1**).

Architectural styles

For this Flood Monitoring and Management System, we implement a combination of Event-Driven Architecture (EDA) and Microservices Architecture. On the one hand, EDA will be a better fit for real-time sensor data processing, threshold-based sensor state change, and triggering alerts for critical events. On the other hand, microservices provide modularity and are better suited for breaking down functionalities into independent services, which enhances scalability and maintainability.

Microservice

To ensure scalability and reduce dependencies among components, the following components are implemented as microservices:

- **Proxy Open Weather API Microservice:** Interfaces with the Open Weather API to fetch weather forecasts and send them to the system for further processing.
- **API for Web/Mobile Applications:** This microservice powers the user interfaces by providing data through API to the stakeholders and manages authentication and access control.
- **Consumer Microservice:** This service (we are using Apache Flink) handles real-time stream processing tasks such as filtering, aggregation, and transformation of incoming data before storing it in the database.
- **Alert Management Microservice:** This service processes data coming from the influxDB database in case there is some critical condition and notifies users through email or SMS.

Event-Driven

Different parts of the system respond based on events. With this implementation, the system allows components to respond dynamically to events in real-time. It also saves energy consumption across the system by utilizing only a portion of devices/software component's capacity or keeping them completely idle depending on the demand. The key elements of the event-driven system for the systems are the following:

- **Message Broker:** Facilitates real-time communication between sensors, gateways, and cloud services by transmitting data streams to the consumer microservice.
- **Open Weather API Integration:** Adjusts sensor data collection frequency and other system behaviors based on weather forecasts, ensuring proactive system optimization.
- **Consumer Microservice:** Processes incoming sensor data streams in real time, performing tasks like anomaly detection, trend analysis, tagging events, and filtering events. This microservice is event-triggered, activating upon receiving new data from the message broker.

- **Alert Management Microservice:** Generates notifications as a form of email and SMS when it receives data with some tag from the InfluxDB database. It generates different kinds of notifications depending on the condition (specified by the tag value), such as sensor malfunctions, adverse weather predictions, and so on.

Architectural Tactics

To ensure the Flood Monitoring and Management System has the required throughput, remains operational during hardware or communication failures, and the data flow remains secure, we will adopt the following tactics:

Performance and Scalability

This project adopts a Stream Processing Architecture using Apache Kafka as the message broker, combined with Apache Flink for real-time data processing. [The choice of Kafka is due to its capabilities to handle high throughput, which makes it durable and ideal for handling large-scale sensor data and analytics. While MQTT is lightweight and efficient, it lacks Kafka's ability to manage persistent streams for advanced analytics which is necessary for this system's performance.](#) Also, it can scale horizontally, which allows the system to accommodate an increasing number of sensors and an increasing number of data streams (in emergency situations, the system will be required to handle increasing number of requests) without significant latency or performance degradation.

Redundancy

Redundancy is ensured at the sensor and subsystem level by having multiple sensors deployed in critical locations. This ensures that if one sensor fails due to environmental conditions, others can continue to provide data. The backup servers are configured to take over if primary servers become unavailable.

Usability

The system is capable of providing real-time data visualization on an interactive dashboard to the different users (managers) at different levels depending on their access level. An intuitive web and mobile applications are made available for non-technical users for ease of usage.

Reliability

Automated failover mechanisms switch from failed components to backup components without human intervention. In addition, continuous health checks allow for real-time monitoring and quick response to any failures.

Load Balancing

Load balancers distribute the incoming data load evenly across available resources. This will prevent any single point of failure due to high loads and ensure that the system can scale efficiently during peak times.

Adaptive System Prioritization

To ensure continuous functionality during severe conditions, the system will implement the following architectural tactics:

- **Prioritized Resource Allocation:** Critical operations, such as water level monitoring and alert generation, will take precedence over less critical tasks like historical trend visualization or non-urgent analytics during resource constraints.
- **Fallback Mechanisms:** In case of partial system failures, the system will automatically switch to reduced functionality modes. For instance, if connectivity is degraded, the system automatically falls back to offline storage on the gateway until communication is restored.

Security

To deal with the security concerns, we implement several strategies that address key security concerns such as data integrity, confidentiality, authenticity, and unauthorized access prevention. Authentication includes multi-factor authentication that enhances the security of the system. Role-Based Access Control will ensure only the authorized user will have access to their respective zones. We will implement TLS (Transport Layer Security) for encrypting communication between different communication points. Also, the software components will reside in cloud platforms that implement the Virtual Private Cloud (VPC) that enhances secure communication among the components.

Sub-systems view

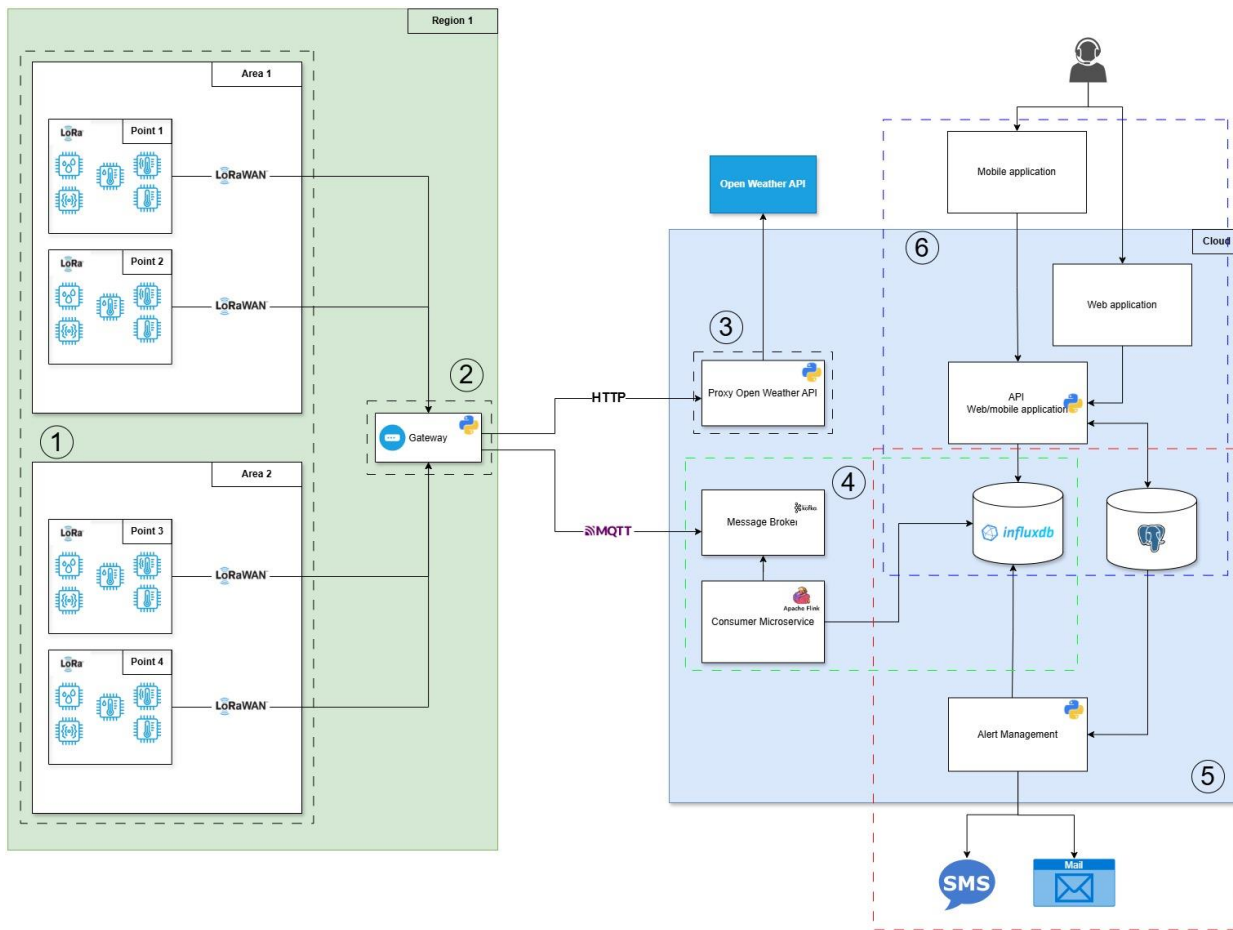


Figure 2 Architectural Sub-System View

In this project, we have identified and divided our architecture design into 6 subsystems.

IoT Data Collection Subsystem (1)

- **LoRaWAN Nodes:** The IoT sensors at points 1-4 are used to track environmental factors that are utilized to forecast the likelihood of flooding. This project's sensor selection is based on the parameters that need to be measured.
- **LoRaWAN Network:** This network uses a low-power wide-area network protocol to link IoT nodes to the gateway.

The main purpose of this subsystem is to collect real-time data from distributed sensor points in different areas.

Gateway Subsystem (2)

- **LoRaWAN Gateway:** The gateway facilitates communication between backend systems and the LoRaWAN network. Data from sensors is compiled and sent to the processing layer. These protocols include the HTTP protocol for integrating with external APIs and the MQTT protocol for transporting messages to the message broker. Functioning as the hub for data relay between cloud-based processing and the Internet of Things network is its primary objective.

Data Processing Subsystem (3)

- **Message Broker:** Ensures dependable data flow by handling asynchronous data streams using a publish-subscribe paradigm.
- **Consumer Microservice:** The microservice handles the broker's incoming data which can involve data transformation, analytics, or additional enrichment.
- **InfluxDB:** A time-series database for effectively storing and retrieving IoT data.

This subsystem ensures robust data processing, storage, and management.

Weather Data Integration Subsystem (4)

- **Proxy Open Weather API:** This API gives users access to outside weather data, which could be used to improve, validate, or compare sensor data that has been gathered. Its primary purpose is to enhance the system's capabilities by adding context or insights relating to weather.

Alert Management Subsystem (5)

- **Alert Management Module:** Queries the database and sends notifications based on predefined conditions (e.g., thresholds, anomalies).
- **Notification Channels:** Sends alerts through SMS and email.
- **PostgreSQL:** Here, PostgreSQL is functional as a relational database that is used to store sensor data, locations, and other operational parameters.
- **InfluxDB:** For important occurrences, like sensor values going over safe limits, the Alert Management Microservice queries InfluxDB. It allows sensor data to be monitored in real-time against preset criteria to escalate replies or send out notifications.

The alert management system ensures proactive communication with stakeholders regarding critical events or situations.

Data Visualization Subsystem (6)

- **Mobile Application:** Provides real-time access to system data and alerts.
- **Web Application:** This is a major part of the system that offers a comprehensive interface for system monitoring and management, usually managed by the system administrators.
- **API Web/Mobile Application:** It acts as an intermediary between the backend and frontend
- **PostgreSQL:** The database stores user information and access permissions, as well as sensors metadata, for web/mobile applications.
- **InfluxDB:** In this subsystem, InfluxDB supports powerful query operations for analyzing real-time data. This serves for operational dashboards and decision support.

The frontend system enables user interaction with the system through intuitive interfaces.

Components, Connectors, and Interfaces View

Components

a. Core Components

- **LoRaWAN Devices:** Sensors/actuators deployed in multiple areas across different regions. These capture and transmit environmental or other operational data.
- **Gateway:** Aggregates and filters data from LoRaWAN devices and forwards it to the backend system over MQTT. Receives Open Weather data from the Open Weather API over HTTP.
- **Consumer Microservice:** Processes data from the message broker for real-time analytics and does needed transformations.
- **Alert Management Service:** Analyzes processed data to trigger alerts via SMS or email.
- **API for Web/Mobile Application:** Provides interfaces for user interaction with the system through the web or mobile applications.
- **Web/Mobile Applications:** User-facing frontends allowing users to visualize data, configure alerts, or perform system management tasks.

b. External Components (Reused/Off-the-Shelf)

- **Open Weather API:** Used for weather data integration, enriching the system's environmental data with external weather forecasts.
- **Apache Kafka Message Broker:** Middleware for asynchronous communication between the gateway and backend processing components.
- **InfluxDB:** Time-series database to store and query sensor data for efficient handling of time-dependent datasets.
- **PostgreSQL Database:** Relational database to store and query users and sensors metadata.
- **Apache Flink:** For stream processing and real-time analytics.
- **SMS/Mail Services:** Third-party integrations (e.g., Twilio, SendGrid) for sending notifications.

Connectors

- **LoRaWAN Communication:** Between the devices and the gateway, using LoRaWAN protocol for low-power, wide-area network communication.
- **MQTT Protocol:** Between the gateway and the message broker, enabling lightweight and reliable message delivery.
- **HTTP Protocol:** For interactions with the Open Weather API and between various backend services.
- **User Interaction:** Web and mobile applications interact with backend APIs using REST or WebSocket protocols.

Interfaces

c. Internal Interfaces

- **Gateway - Apache Kafka Message Broker**
- **Interface Type:** Data Publishing Interface
- **Description:** Gateways equipped with edge computing capabilities publish preprocessed sensor data to Kafka topics via a well-defined protocol (e.g., MQTT). Data includes sensor measurements, operational status, and metadata. The interface ensures data integrity, supports configurable retry mechanisms for transient failures, and leverages Kafka's partitioning for scalability and load balancing.
- **Apache Kafka Message Broker - Consumer Microservice**
- **Interface Type:** Data Subscription Interface
- **Description:** The Consumer Microservice subscribes to specific Kafka topics to process data streams. The interface supports efficient message retrieval using Kafka's offset tracking and allows the consumer to operate in real-time or batch mode. Error handling and dead-letter queues are implemented to handle processing failures.
- **Consumer Microservice – InfluxDB**
- **Interface Type:** Data Insertion Interface
- **Description:** The microservice writes processed time-series data to InfluxDB using its native HTTP API or client libraries. The interface supports high throughput writes, ensuring timestamp accuracy and data consistency for queries by downstream services.
- **Alert Management Service – InfluxDB**
- **Interface Type:** Data Query Interface
- **Description:** The Alert Management Service queries time-series data from InfluxDB using its Flux query language. Queries are optimized for retrieving specific sensor metrics and trend analysis to detect anomalies or threshold breaches efficiently.
- **Web/Mobile Application API – InfluxDB**

- **Interface Type:** Data Query Interface
- **Description:** The API fetches data from InfluxDB to serve user interfaces. It supports customizable queries based on user access levels (e.g., area-specific or regional views) and filters for temporal granularity.
- **Alert Management Service**
- **Interface Type:** Metadata Query Interface
- **Description:** This interface allows the Alert Management Service to query metadata about users, roles, and sensors from PostgreSQL. Metadata includes sensor configurations, assigned areas, and user contact information, used to personalize and route alerts.
- **Web/Mobile Application API – PostgreSQL**
- **Interface Type:** Metadata Query Interface
- **Description:** The API retrieves user and sensor metadata from PostgreSQL to authenticate users, determine access rights, and provide contextual information in the UI. Queries are optimized for low latency to ensure a responsive user experience.

d. External Interfaces

- **Open Weather API**
- **Interface Type:** Data Retrieval Interface
- **Description:** The system fetches weather forecast data via HTTP RESTful requests to the Open Weather API. The interface supports authentication through API keys, handles rate limits, and integrates weather data into the predictive analytics pipeline.
- **Web/Mobile Application API**
- **Interface Type:** User Interaction Interface
- **Description:** The API provides RESTful endpoints for user-facing applications to interact with the backend. Endpoints include functionalities for user authentication, role management, data visualization, and alert management. The interface supports JSON payloads and ensures secure communication using HTTPS.
- **Notification Services**
- **Interface Type:** Communication Interface
- **Description:** Interfaces for sending notifications leverage external services like Twilio for SMS and SendGrid for email. The integration uses their respective RESTful APIs, handling message templates, contact lists, and delivery statuses. Error handling ensures retry mechanisms for undelivered messages.

System boundaries

e. Inside the System

- All LoRaWAN devices, the gateway, and backend services (message broker, consumer microservice, API services, alert management) are within the system's scope of control.
- Web and mobile applications are part of the system

f. Outside the System

- **Open Weather API**
- **Notification Providers (SMS/Mail)**
- **External devices** (e.g., phones, computers) accessing web and mobile applications are outside the system boundary.

CAPS view

[CAPS D1, CUPCARBON D2]

In this section, the CAPS SAML view of a subset of our system is presented. The image is also available in the GitHub repository at the following [link](#).

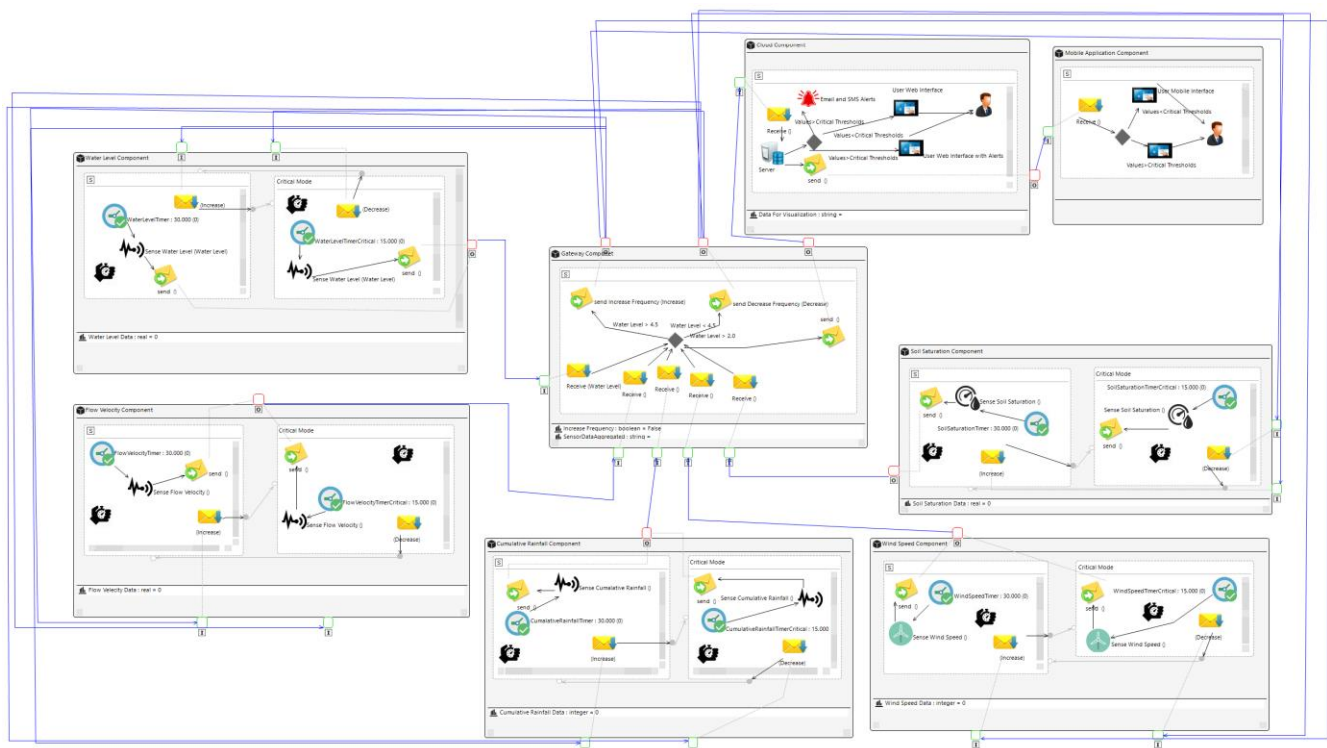


Figure 3 CAPS view of the system

The SAML CAPS representation of our system provides a detailed depiction of the IoT subsystem, the edge computing gateway, and the cloud infrastructure that interfaces with users via web and mobile applications. The system is also equipped with an alerting mechanism for critical notifications. The IoT

sensors operate in two distinct modes: critical and normal. These modes are dynamically activated or deactivated depending on the water level. For instance, when the water level drops below a certain threshold, data transmission to the cloud is halted, as such information is deemed irrelevant for potentially dangerous scenarios. The edge gateway aggregates data to provide a snapshot of the current state. The aggregated data is then transmitted to the cloud infrastructure, which forms the central hub for user interaction and alerting. In the cloud layer, user interfaces are made available through web and mobile applications. These interfaces allow users to monitor the floods states and receive notifications in real time. Alerts are prominently displayed in the interface when critical thresholds are breached. Furthermore, the system enhances user engagement by sending critical notifications through SMS and email.

CUPCARBON view

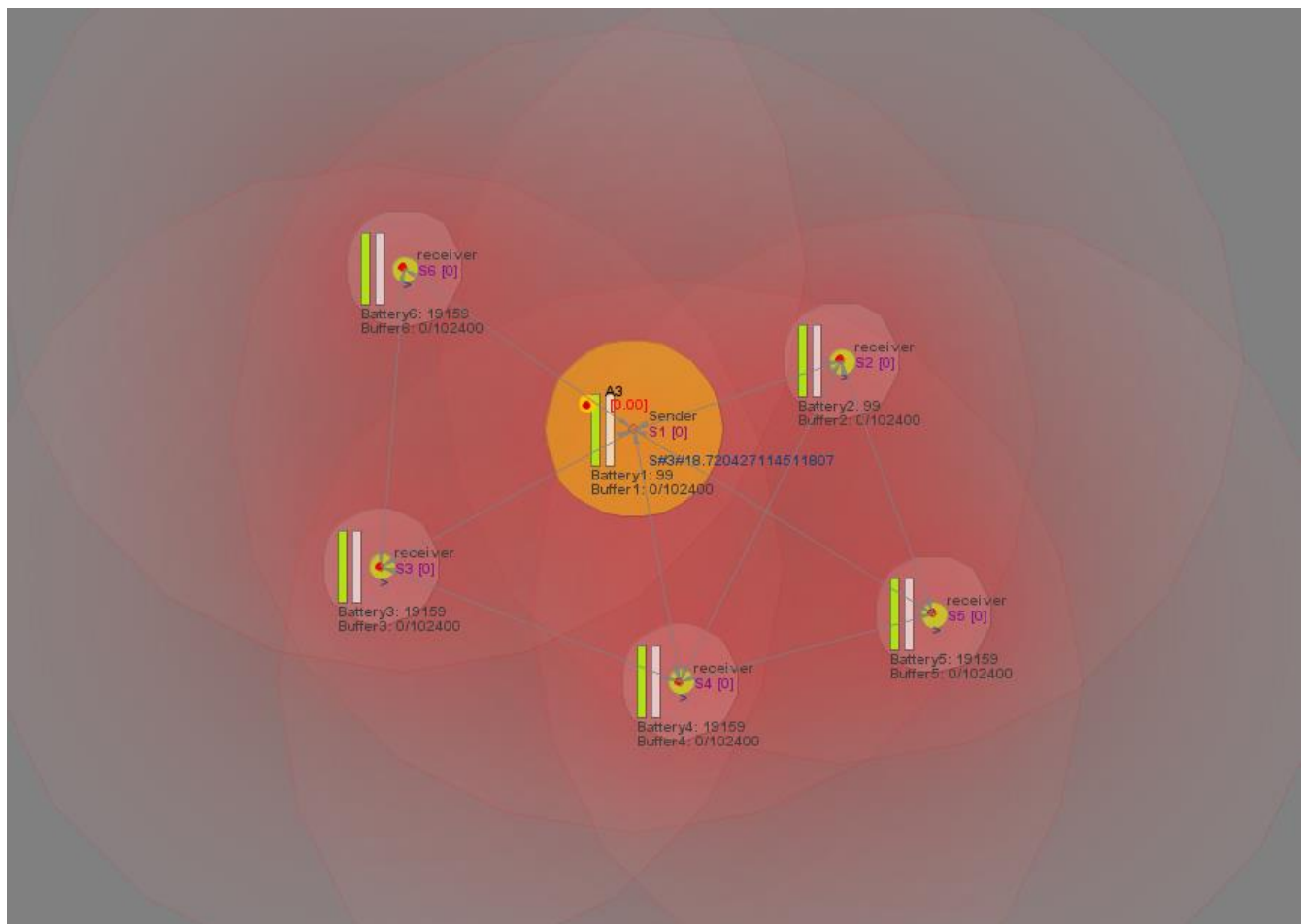


Figure 4 CUPCARBON view of the system

The CupCarbon representation of our system provides a high-level overview of the 5 basic sensing components as shown in the CAPS view. We have imported the .capssaml file from the CAPS project to measure the battery consumption of the sensors involved in the project. As seen from the diagram, each sensing unit's battery consumption is displayed which can be adjusted to get the simulation over a longer period of time. This aims to simulate the amount of time the sensors can work before running

out of battery life and measure their energy efficiency.

Architecture Decision Record

In this section, we report a table to represent the chosen option between the options given by the model. To read the complete reports, please refer to the GitHub repository on this [link](#)

	Alessandro			Raluca Mihaela			Md Abdus			Salim		
Context	0 shot	RAG	Fine tuned	0 shot	RAG	Fine tuned	0 shot	RAG	Fine tuned	0 shot	RAG	Fine tuned
1	Accepted	Accepted	Declined	Accepted	Accepted	Declined	Accepted	Accepted	Declined	Accepted	Accepted	Declined
2	Accepted	Superseded	Declined	Accepted	Superseded	Declined	Accepted	Accepted	Declined	Accepted	Accepted	Declined
3	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted	Accepted
4	Accepted	Accepted	Declined	Accepted	Accepted	Accepted	Accepted	Accepted	Declined	Accepted	Accepted	Declined
5	Accepted	Accepted	Declined	Accepted	Accepted	Declined	Accepted	Accepted	Declined	Accepted	Accepted	Declined

Legends:

Accepted

Superseded

Declined

Architecture Design Decisions [for deliverables D2]

Select the three most important Architecture Design Decisions related to deliverable D2. Document them using the QoC excel file in Teams.

Architectural Design Decision 1 – Selecting Sensors

Concern (Identifier: Description)		Con#1: What sensor technology should be used for monitoring environmental parameters?
Ranking criteria (Identifier: Name)		Cr#1: Communication range Cr#2: Power efficiency Cr#3: Cost
Options	Identifier: Name	Con#1-Opt#1: LoRaWAN Sensors
	Description	LoRaWAN sensors are low-power, long-range communication sensors designed for IoT systems in remote or urban areas.
	Status	This option is decided.
	Relationship(s)	This decision is related to Con#3-Opt#2: LoRaWAN Gateway
	Evaluation	Cr#1: LoRaWAN sensors excel in long-range communication, making them suitable for monitoring large, remote flood-prone areas. Cr#2: They are highly power-efficient, reducing the need for frequent battery replacement. Cr#3: LoRaWAN sensors are moderately priced, balancing cost and performance effectively.
	Rationale of decision	This option was chosen because the combination of long communication range and power efficiency outweighs the moderate cost.
	Identifier: Name	Con#1-Opt#2: Zigbee Sensors
	Description	Zigbee sensors are low-power, short-range communication sensors commonly used in IoT systems.
	Status	This option been rejected
	Relationship(s)	-
	Evaluation	Cr#1: The short communication range limits their use in large or remote areas. Cr#2: Power efficiency is good, but it does not compensate for the limited range. Cr#3: Zigbee sensors are cost-effective but unsuitable for the FMMS use case due to communication limitations.
	Rationale of decision	This option is rejected because the limited communication range (Cr#1) does not meet the requirements for FMMS.

References:

- [1] Remco C. de Boer, Rik Farenhorst, Patricia Lago, Hans van Vliet, Viktor Clerc, Anton Jansen, ""Architectural Knowledge: Getting to the Core"", in 3rd International Conference on the Quality of Software-Architectures (QoSA), Boston, USA, July 12-13 2007.
- [2] P. Kruchten, ""An Ontology of Architectural Design Decisions in Software-Intensive Systems,"" in 2nd Groningen Workshop on Software Variability Management, Groningen, The Netherlands, 2004.

Architectural Design Decision 2 – Selecting the Message Brocker

Concern (Identifier: Description)		Con#2: Which message broker should be used for handling sensor data?
Ranking criteria (Identifier: Name)		Cr#1: Throughput Cr#2: Fault tolerance Cr#3: Scalability
Options	Identifier: Name	Con#2-Opt#1: Kafka
	Description	Kafka is a distributed, high-throughput message broker that supports durable and scalable data streaming
	Status	This option been decided
	Relationship(s)	Indicate relationships with other options (by using their identifiers): {forbids, conflicts with, enables, subsumes, is related to}
	Evaluation	Cr#1: Kafka offers high throughput, ideal for processing large-scale sensor data. Cr#2: Its fault-tolerant architecture ensures reliability even in case of node failures. Cr#3: Kafka scales easily with increasing data loads, supporting future expansion of the FMMS.
	Rationale of decision	Kafka is chosen because its combination of high throughput, fault tolerance, and scalability supports the system's reliability and future growth needs.
	Identifier: Name	Con#2-Opt#2: MQTT
	Description	MQTT is a lightweight, low-overhead protocol designed for IoT communication
	Status	This option been rejected
	Relationship(s)	-
	Evaluation	Cr#1: MQTT has moderate throughput, limiting its effectiveness in large-scale systems. Cr#2: Fault tolerance is limited compared to Kafka, as MQTT brokers do not inherently support distributed architecture. Cr#3: MQTT is lightweight and suitable for simple IoT use cases but lacks scalability for FMMS needs.
	Rationale of decision	This option is rejected because its limited throughput and scalability (Cr#1, Cr#3) make it unsuitable for handling the large-scale data requirements of FMMS.

[1] Remco C. de Boer, Rik Farenhorst, Patricia Lago, Hans van Vliet, Viktor Clerc, Anton Jansen, "Architectural Knowledge: Getting to the Core", in 3rd International Conference on the Quality of Software-Architectures (QoSA), Boston, USA, July 12-13 2007.

[2] P. Kruchten, "An Ontology of Architectural Design Decisions in Software-Intensive Systems," in 2nd Groningen Workshop on Software Variability Management, Groningen, The Netherlands, 2004.

Architectural Design Decision 3 – Selecting Gateway

Concern (Identifier: Description)		Con#3: Which gateway should be used for integrating sensors and backend systems?
Ranking criteria (Identifier: Name)		Cr#1: Compatibility with sensors Cr#2: Ease of deployment Cr#3: Cost
Options	Identifier: Name	Con#3-Opt#1: LoRaWAN Gateway
	Description	A LoRaWAN gateway aggregates data from LoRaWAN sensors and bridges it to the backend system.
	Status	This option been decided
	Relationship(s)	This decision is related to Con#1-Opt#1: LoRaWAN Sensors.
	Evaluation	Cr#1: LoRaWAN gateways are highly compatible with LoRaWAN sensors, ensuring seamless integration. Cr#2: Deployment is moderately complex but manageable with proper planning. Cr#3: Cost is moderate, balancing functionality and reliability effectively.
	Rationale decision	of This option is chosen because it provides seamless compatibility with LoRaWAN sensors and ensures reliable communication over long distances.
	Identifier: Name	Con#3-Opt#2: Generic MQTT Gateway
	Description	A generic MQTT gateway supports MQTT communication for IoT sensors and devices.
	Status	This option been rejected
	Relationship(s)	-
	Evaluation	Cr#1: Limited compatibility with LoRaWAN sensors, requiring additional integration effort. Cr#2: Deployment is straightforward but does not align with the chosen sensor technology. Cr#3: Cost is low, but it does not justify the lack of compatibility.
	Rationale decision	of This option is rejected because the limited compatibility with LoRaWAN sensors (Cr#1) outweighs the benefits of ease of deployment and low cost.

Architectural Design Decision 4 – Deployment and Scalability

Concern (Identifier: Description)		Con#4: How should the system be deployed to ensure ease of deployment and scalability?
Ranking criteria (Identifier: Name)		Cr#1: Portability Cr#2: Scalability Cr#3: Ease of integration
Options	Identifier: Name	Con#4-Opt#1: Docker Containers with Docker Compose
	Description	Use Docker containers for each component of the system (e.g., sensors, message broker, database, visualization) with Docker Compose to orchestrate multi-container deployments.
	Status	This option been decided
	Relationship(s)	Related to Con#2-Opt#1: Kafka and Con#3-Opt#1: LoRaWAN Gateway.
	Evaluation	Cr#1: Containers ensure portability across different environments, allowing the system to run consistently on any machine with Docker installed. Cr#2: Docker Compose enables adding or scaling containers easily, supporting system growth without major reconfiguration. Cr#3: Integration is simplified as containers communicate through predefined networks and are configured via Docker Compose files.
	Rationale decision of	This option is chosen because it offers an optimal balance of portability, scalability, and ease of integration, supporting the system's long-term deployment needs.
	Identifier: Name	Con#4-Opt#2: Virtual Machines (VMs)
	Description	Use traditional Virtual Machines for deploying system components, with each VM running specific services (e.g., sensor simulators, databases, brokers).
	Status	This option been rejected
	Relationship(s)	-
	Evaluation	Cr#1: VMs provide portability to some extent, but they require more resources and are less flexible than containers. Cr#2: Scalability is limited as VMs are heavier and take longer to spin up compared to containers. Cr#3: Integration is more complex due to the need for additional configuration for inter-VM communication.
	Rationale decision of	This option is rejected because the limited scalability (Cr#2) and higher complexity of integration (Cr#3) make it unsuitable for the dynamic and scalable requirements of the system.
	Identifier: Name	Con#4-Opt#3: Bare-metal Deployment
	Description	Deploy the system directly on physical machines without virtualization or containerization.
	Status	This option been rejected
	Relationship(s)	

	Evaluation	Cr#1: Bare-metal deployment lacks portability as the system depends heavily on the underlying hardware and OS configuration. Cr#2: Scalability is extremely limited as adding new components requires significant manual effort. Cr#3: Integration is cumbersome, requiring manual setup of dependencies and communication between components.
	Rationale decision	This option is rejected because it fails to meet the portability (Cr#1) and scalability (Cr#2) requirements critical for the system.

[1] Remco C. de Boer, Rik Farenhorst, Patricia Lago, Hans van Vliet, Viktor Clerc, Anton Jansen, ""Architectural Knowledge: Getting to the Core"", in 3rd International Conference on the Quality of Software-Architectures (QoSA), Boston, USA, July 12-13 2007.

[2] P. Kruchten, ""An Ontology of Architectural Design Decisions in Software-Intensive Systems,"" in 2nd Groningen Workshop on Software Variability Management, Groningen, The Netherlands, 2004.

Pitch of your proposal [for deliverable D2]

Please submit a 3-minute pitch of your proposal.

The pitch can be found [here](#)

Prototype Description [for deliverable D2]

Do not simply attach the code. Please describe which part of the system you are prototyping and explain how it supports your claims about your project's functional and non-functional requirements. The prototype must demonstrate that your architecture, when implemented, brings the quality characteristics you wanted/expected.

Be sure to attach a video of the working prototype and a simple explanation.

The prototype of the project covers a few areas focusing mainly on generating high-volume sensor data, processing them in a gateway, and sending them for further processing, analysis, and visualization. These are the core and most critical parts of our architecture. This prototype includes an end-to-end data flow from data generation to visualization for the end user.

What does this prototype demonstrate?

This prototype is designed to demonstrate the following scenario such as

- **Producing a high volume of sensor data:**
The prototype simulates more than 150,000 data (message) per minute using a python script and sends them to the gateway (Apache Flink) using an MQTT protocol.
- **Real time high-volume data processing:**
An MQTT consumer consumes data for a specific topic and this data is being processed. This processed data is then passed to Kafka by using Kafka Producer. Because the data volume is high, and cannot be efficiently processed by a single function, we implemented Flink to ensure scalability.
- **Data Storage and visualization:**

This prototype implements a Kafka broker and acts as a central message broker. Telegraf acts as a Kafka consumer and sends the data to InfluxDB and our Web UI shows real-time visualization that built on top of Grafana.

- **User Access and Sensor Management:**

The developed UI provides the functionality to see the sensors details and provides the user access management.

- **Satisfying functional and non-functional requirements:**

The prototype demonstrates it satisfies few functional and non-functional requirements as this can ingest a high volume of data and process them in real time and has a feature to visualize them alongside providing user access and sensor management in a web UI.

How would this prototype satisfy functional and non-functional requirements and bring the required quality characteristics?

The core consideration while making this prototype was to ensure high-volume data generation and processing in real-time. This is the core functionality of the system that needs to work without any problem so that every other component works correctly.

- This implementation of the prototype can generate the required number of data (150,000 messages/min). This ensures a high throughput sensor data generation (FR1, NFR1). And each sensor has its own identity to make it traceable (FR7)
- The gateway part of the prototype can handle high volume incoming sensor data, process, and send them to the Kafka Broker in real-time with very low latency with batch processing. This also ensures high throughput performance (FR6, NFR1) and since the messages are batched and they are acknowledged by the Kafka Broker, it ensures reliability (NFR2).
- The time series database handles many data streams, and the web UI successfully shows the visual representation. Resembles it aligns with several functional and non-functional requirements, such as FR1, NFR1.
- In Webapp endpoint, user and sensor management is implemented that satisfies FR8, FR9, and FR10. It has multiple levels of user management that satisfies FR14.
- The whole prototype is implemented as microservices that can be scaled up and down, making it available for higher workloads (NFR3).
- Security is implemented in every segment of the prototype using password or token-based authentication; therefore, the prototype can be called secured; and therefore, satisfies NFR4.

The demonstration video of this prototype can be found [here](#)

Implementation Architecture [only for Deliverable D2]

Please report here the technical/implementation architecture you are using for realizing your prototype. We expect here an informal, box&line description of the technological components you are using to implement your conceptual architecture presented in the previous section.

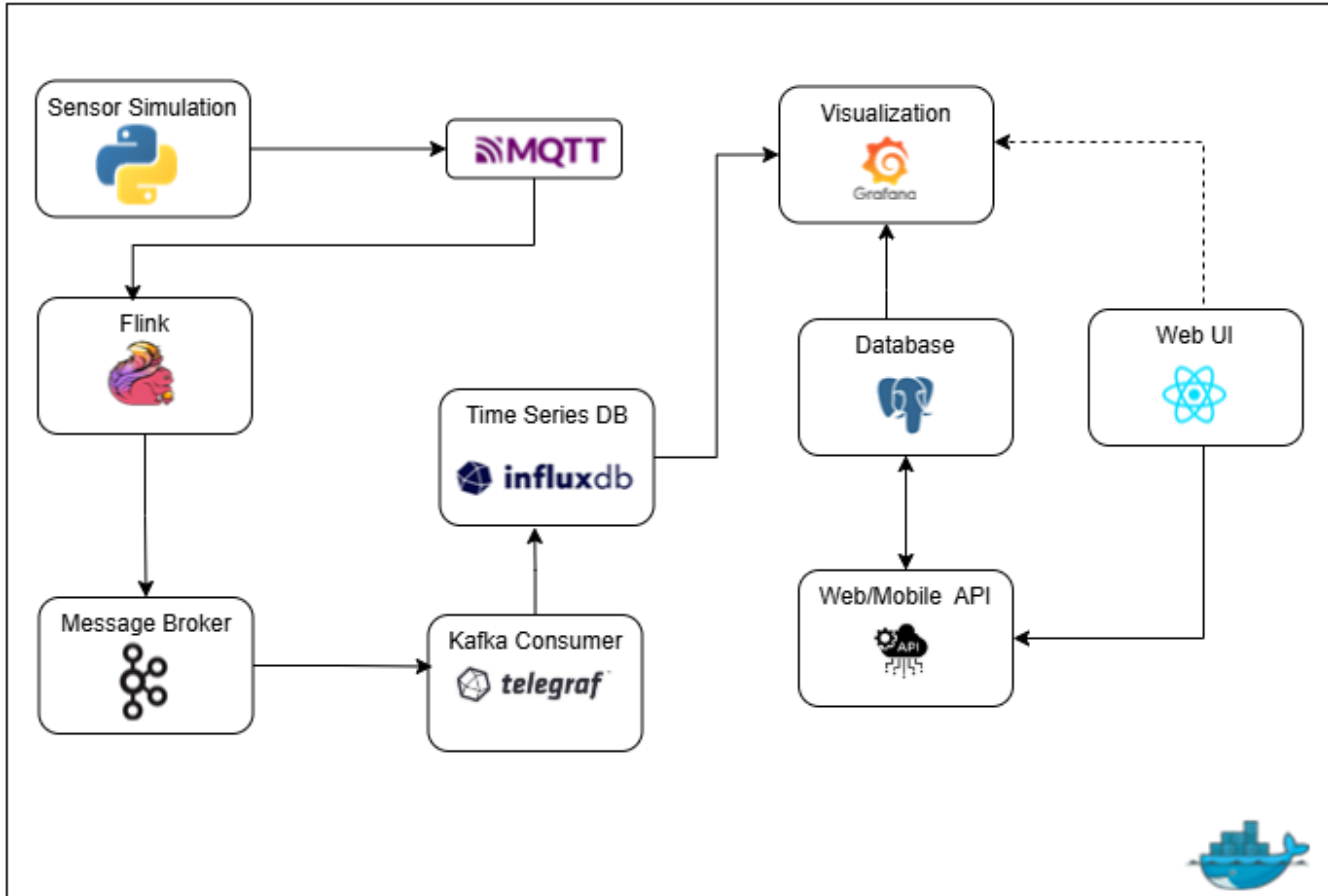


Figure 5 Architectural Implementation

Implementation Architecture: This architecture implementation leverages containerized microservices to ensure scalability, portability, and modularity. The architecture includes:

- **Sensor Simulation (Python):** Simulates IoT sensors to generate data streams for water level, river flow velocity, cumulative rainfall, wind speed, and soil saturation.
- **MQTT Broker:** It serves as a lightweight communication protocol used for real-time, topic-based message transfer from the sensor simulation to the gateway.
- **Gateway (PyFlink):** Processes incoming MQTT data by transforming and preparing it into structured JSON messages before pushing them to Apache Kafka.
- **Message Broker (Apache Kafka):** This microservice ensures reliable and scalable delivery of processed messages to downstream consumers.
- **Telegraf:** Telegraf acts as a data collection and forwarding agent consuming data streams from Kafka topics, processes or transforms the data, and efficiently writes the resulting data into InfluxDB. It provides robust buffering and retry mechanisms to handle intermittent connectivity issues, ensuring

more reliable data flow and reducing the risk of data loss.

- **Time-Series Database (InfluxDB):** Stores sensor data for real-time querying and visualization which is accessed by Grafana for advanced analytics and visualization.
- **System Database (PostgreSQL):** Stores metadata, user configurations, and system logs for administrative purposes. This database also stores sensors for when there is a need to locate any of the sensor locations for maintenance purposes. As seen from the implementation architecture, it shows a two-way communication because it also saves user information when a new sensor or manager is added from the web UI.
- **Visualization System (Grafana):** The Grafana is shown as an extension from the web UI as it provides different visualization capabilities from the web UI which depends on the administrative privilege assigned to the user. It displays real-time and historical data via interactive dashboards ensuring each regional manager only has access to the data specific to his region.
- **Web UI (React Native):** Provides endpoints for external systems or users to access the data stored in InfluxDB and PostgreSQL. The highest ranked administrator can view dashboards from all regions, add new sensors, and users from the web UI.

The entire architecture is deployed using Docker to simplify management, ensure consistency and scalability, and enable portability.

“Demonstration” of your architecture compliance to Functional and Non-Functional Requirements [only for Deliverable D2]

Compliance to Functional Requirements

The following demonstrates the project compliance to the functional requirements of the Flood Monitoring Management System.

- **Real-time Data Visualization:** Environmental parameters such as water levels, river flow velocity, rainfall, soil saturation, wind speed, and direction are sent from the sensors, processed by gateway and goes through message broker and stored in InfluxDB which is a time-series database. Grafana has been connected to InfluxDB to retrieve data and display with continuous updates for real-time dashboards. This ensures users (managers) can view changes in environmental conditions in real-time, fulfilling the requirement for real-time data visualization. Likewise, thresholds for environmental parameters are defined within the system such that when sensor readings exceed a set threshold, alerts are triggered and communicated through Kafka which is displayed on the Grafana dashboard. This ensures area managers to be promptly informed when safety thresholds are breached.
- **Geographic Monitoring:** The system allows sensors to be tagged based on their geographic locations which has been used in form of areas in this project. Managers can filter and select

specific areas through the React Native dashboard, providing the capability to monitor specific areas. These tags, combined with the database's querying capabilities, enable geographic-based monitoring seamlessly.

- **Sensor Malfunction Alerts:** The system monitors sensor activity by checking for regular data input and collecting current sensor information in the PostgreSQL database. This information is regularly sent alongside environmental parameters; If a sensor fails to send data within a defined interval, a sensor malfunction alert is generated. These alerts are prioritized based on the sensor's criticality, ensuring issues with critical sensors are addressed promptly.
- **Priority Alarms for Imminent Danger:** For imminent threats like potential flooding, the system prioritizes alarms by analysing aggregated sensor data and triggering immediate notifications when conditions indicate high-risk scenarios. These alarms are visually distinct on dashboards and are delivered via integrated communication channels to ensure swift action.

Compliance to Non-Functional Requirements

1. Scalability

Modularity and Scaling: with microservice architecture implementation, all components of the system were designed in form of containers in Docker which makes it easier to scale/allocate additional resources independently.

Distributed Messaging: Message broker and time series database like Kafka and InfluxDB performance can be tuned to utilize available hardware efficiently and configured to process and store large volumes of data. Also, InfluxDB supports clustering for distributed storage which makes it a better option for the FMMS project.

2. Reliability

The system has adopted several strategies from redundant components, sensor information monitoring, load balancing amongst many others to ensure the reliability of the system. Some of these include:

- Kafka replication is used to provide fault tolerance, and the acknowledgement functionality is used to ensure minimum data loss.
- MQTT brokers can run in clustered or load-balanced configurations.
- Docker Compose can restart failed services automatically without manual intervention.
- An area manager can replace faulty sensors to ensure continuous data retrieval.

3. Performance

- Kafka has been used because it supports high throughput for message processing.
- InfluxDB is optimized for high read and write rates.

- Flink provides scalability and it can process a very high volume of data, that makes our prototype robust to handle high data rate when necessary.
- The Kafka Producer can batch process messages to reduce overhead. Flink ensures that the functions are not overloaded by the request by ensuring proper scalability.

4. Security

Data Generation: LoRaWAN Sensor has been used in this system to provide secure communication, leveraging its built-in AES-128 encryption for data transmission.

Data Transmission and storage: The system provides secure data transmission and robust authentication/authorization through MQTT which uses username/password authentication and TLS for secure transmission. Likewise, Kafka and InfluxDB can use SSL/TLS and basic authentication. For additional security, a login page has been designed using React Native for managers to provide an additional layer of security and data access levels before gaining access to the Grafana dashboard and user management console.

5. Usability

To ensure ease of usage, an intuitive User Interface has been designed which is tailored for different user levels with clear navigation. This can be accessed from both desktop and mobile application to view interactive dashboards and manage sensor and user information.

6. Maintainability

To ease the maintainability of the whole system, we have employed the microservice architecture which allows independent updates, repair, replacement and scaling of components with minimal downtime. Additionally, all components were containerized using docker to ensure quick deployment and updates.

7. Energy Efficiency

- **Low Power Sensors:** LoRaWAN sensors operate in low-power modes, while efficient processing and load balancing minimize overall energy usage.
- **Low power communication protocol:** MQTT is lightweight, reducing energy use during communication.

Summary [only for Deliverable D2]

In this section, you should describe how the addressed design topic has been expressed in the various parts of the document.

The Flood Management and Monitoring System (FMMS) is an Internet of Things (IoT) solution that uses energy-efficient sensors to track water level, river flow velocity, cumulative rainfall, wind speed, and soil saturation to efficiently notify users of the possibility of floods in flood prone areas. The data is stored in InfluxDB for high-performance retrieval after being safely sent via MQTT through Pyflink to a Kafka-based message broker. On the web UI interface, the Grafana dashboard is hosted to display real-time trends, guaranteeing that only authorized individuals can view vital information. Docker Compose has been used to containerize the system, allowing for smooth deployment and scaling. The FMMS optimizes energy consumption while satisfying important non-functional needs including scalability, performance, and security. By offering fast alerts and actionable data for efficient decision-making, this technology improves flood preparedness and reduces the effect of flood on the environment.